

Threads Priorities in C# with Examples

In this article, I am going to discuss **Threads Priorities in C#** with Examples. Please read our previous article where we discussed [Thread Life Cycle in C#](#) with Examples.

Threads Priorities in C#

In C# Programming Language, each and every thread has a priority that determines how often the thread gets access to the CPU. In general, a Low-Priority Thread will get less CPU time than a High-Priority Thread. The important point that we need to understand is how much CPU time a thread will get, it doesn't only depend on its priority, but also depends on the kind of operation it is performing.

For example, if a high-priority thread is waiting for some shared I/O resources to complete its task, then it will be blocked and taken off by the CPU. And at the same time, a lower-priority thread may get the CPU time and finish its execution if it doesn't require such shared I/O resources. In scenarios like this, a High-Priority Thread may get less CPU time than a Low-Priority Thread over a specific period of time.

Another factor that determines how much CPU time is allocated to a thread is how the task scheduling is implemented by the operating system.

How to Set the Priority of a Thread in C#?

When we created an instance of the Thread class, the thread object gets a default priority setting. We can get or set the Priority of a thread by using the following Priority property of the Thread class.

1. **ThreadPriority Priority {get; set;}:** This property is used to get or set a value indicating the scheduling priority of a thread. It returns one of the ThreadPriority values. The default value is ThreadPriority.Normal.

That means we can set the Priority property with one of the fields of the ThreadPriority Enum. If you go to the definition of the ThreadPriority enum. Then you will see the following signature.

```
namespace System.Threading
{
    ...public enum ThreadPriority
    {
        ...Lowest = 0,
        ...BelowNormal = 1,
        ...Normal = 2,
        ...AboveNormal = 3,
        ...Highest = 4
    }
}
```

The ThreadPriority enum provides the following 5 properties:

1. **Lowest = 0**: The Thread can be scheduled after threads with any other priority. That means threads with the Lowest Priority can be scheduled after threads with any other higher priority.
2. **BelowNormal = 1**: The Thread can be scheduled after threads with Normal priority and before those with Lowest Priority. That means threads with BelowNormal priority can be scheduled after threads with Normal priority and before threads with Lowest priority.
3. **Normal = 2**: The Thread can be scheduled after threads with AboveNormal priority and before those with BelowNormal Priority. Threads have Normal priority by default. That means threads with Normal priority can be scheduled after threads with AboveNormal priority and before threads with BelowNormal and Lowest priority.
4. **AboveNormal = 3**: The Thread can be scheduled after threads with the Highest priority and before those with Normal Priority. That means threads with AboveNormal priority can be scheduled after the thread with the Highest priority and before threads with Normal, BelowNormal, and Lowest priority.
5. **Highest = 4**: The Thread can be scheduled before threads with any other priority. That means threads with the Highest Priority can be scheduled before threads with any other priority.

Note: By default, when we created a thread, it gets a default priority of 2 i.e. ThreadPriority.Normal

How can we Set and Get the Priority of a Thread in C#?

Let us see an example to understand how to set and get the priorities of a Thread in C# by using the Priority property of the Thread class. For a better understanding, please have a look at the below example.

```
using System;
using System.Threading;
namespace ThreadStateDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Thread thread1 = new Thread(SomeMethod)
            {
                Name = "Thread 1"
            };
            //Setting the thread Priority as Normal
            thread1.Priority = ThreadPriority.Normal;
            Thread thread2 = new Thread(SomeMethod)
            {
                Name = "Thread 2"
            };
            //Setting the thread Priority as Lowest
            thread2.Priority = ThreadPriority.Lowest;
            Thread thread3 = new Thread(SomeMethod)
            {
                Name = "Thread 3"
            };
            //Setting the thread Priority as Highest
            thread3.Priority = ThreadPriority.Highest;
```

```

        //Getting the thread Priority
        Console.WriteLine($"Thread 1 Priority: {thread1.Priority}");
        Console.WriteLine($"Thread 2 Priority: {thread2.Priority}");
        Console.WriteLine($"Thread 3 Priority: {thread3.Priority}");
        thread1.Start();
        thread2.Start();
        thread3.Start();
        Console.ReadKey();
    }
    public static void SomeMethod()
    {
        for (int i = 0; i < 3; i++)
        {
            Console.WriteLine($"Thread Name: {Thread.CurrentThread.Name} {i}");
        }
    }
}

```

Output:

```

Thread 1 Priority: Normal
Thread 2 Priority: Lowest
Thread 3 Priority: Highest
Thread Name: Thread 3 Printing 0
Thread Name: Thread 3 Printing 1
Thread Name: Thread 3 Printing 2
Thread Name: Thread 1 Printing 0
Thread Name: Thread 2 Printing 0
Thread Name: Thread 2 Printing 1
Thread Name: Thread 2 Printing 2
Thread Name: Thread 1 Printing 1
Thread Name: Thread 1 Printing 2

```

Note: The output is unpredictable because threads are highly system-dependent. The thread scheduler of the operating system schedule threads with no guarantee but tries to consider them. For long-running task threads get the benefit of priority setting.

Why do we need Thread Priority in C#?

Well, this is not required in common cases. However, in some cases where you may want to elevate the priority of some threads. One such example could be when you want certain tasks to be completed first over others.

Points to Remember:

1. A programmer can explicitly assign priority to a thread.

2. The default value is `ThreadPriority.Normal`
3. The operating system does not assign priority to threads.
4. It will throw `ThreadStateException` if the thread has reached a final state, such as `Aborted`.
5. It will throw `ArgumentException` if the value specified for a set operation is not a valid `ThreadPriority` value.
6. It is not guaranteed that the thread whose priority is high will execute first and the thread whose priority is low will execute after. Due to context switching the highest priority thread may execute after the lowest priority thread.

In the next article, I am going to discuss [How to Terminate a Thread in C#](#) with Examples. Here, in this article, I try to explain **Threads Priorities in C#** with Examples. I hope you enjoy this Thread Priority in C# with Examples article.